

# YASMIN: A TINY PERFORMANCE-PORTABLE STENCIL DSL FOR WEATHER AND CLIMATE KERNELS

*Jonathan Fornera, Saskia Koller, Sascha Pucillo*

High Performance Computing for Weather and Climate  
ETH Zurich, Switzerland

## ABSTRACT

Stencil computations are the computational core of structured-grid weather and climate models, yet writing a numerical kernel once for every target architecture (CPUs, multi-core CPUs, and GPUs) is both error-prone and a poor use of domain-scientist time. We present YASMIN (Yet Another Stencil Mini Intermediate Notation), a small embedded Python DSL that lets users define stencil operators, select a backend, and let Yasmin handle everything else. Yasmin lowers the stencil operator through two intermediate representations (IRs): a Stencil IR that keeps array accesses relative and symbolic, and a Loop IR that fully decomposes them into explicit index arithmetic for backends without native array slicing. Yasmin provides NumPy, native C++, and OpenMP C++ execution backends. OpenMP code generation is shape-aware and can adapt the thread count and loop-collapsing decision to the concrete iteration domain, while scheduling remains configurable.

We evaluate Yasmin on a five-point Laplacian and an advection–diffusion stencil against hand-written implementations and GT4Py. The NumPy backend shows some abstraction overhead, while the generated C++ and OpenMP backends remain competitive and scale well across multiple cores.

## 1. INTRODUCTION

**Stencil computations.** Stencil computations are a class of numerical operations in which the value at each grid point is computed from the value at that point and a fixed set of neighboring grid points. They are particularly common in numerical weather and climate models, where spatial derivatives, diffusion, and advection operators are repeatedly applied to fields defined on structured grids.

Because stencil accesses can extend beyond the point currently being updated, not every grid point can necessarily be evaluated without additional boundary data. In a local computation, the valid execution domain is the subset of grid points for which all required accesses are available. In distributed implementations, neighboring data is commonly

stored in a surrounding *halo* region and exchanged between adjacent subdomains [1].

**Motivation.** The regular and predictable access patterns of stencil computations make them well suited to parallel execution and automatic code generation. Since weather and climate models repeatedly apply such computations over large grids and many time steps, the performance of individual stencil kernels can substantially affect overall runtime.

This motivates domain-specific languages (DSLs) such as Yasmin and GT4Py [2], which separate the mathematical stencil definition (the *what*) from its architecture-specific execution strategy (the *how*). Domain scientists can therefore focus on the numerical method, while the DSL represents the computation in an intermediate representation (IR) and generates target-specific code.

**Related Work.** GT4Py is a Python framework designed for high-performance stencil computations in weather and climate applications [2]. It uses an embedded DSL and code-generation toolchain to separate the numerical definition of a stencil from its target-specific implementation. GT4Py is therefore closely related to Yasmin and serves as the principal DSL baseline in our evaluation.

Devito is an embedded Python DSL for finite-difference PDE solvers [3]. It uses symbolic mathematical expressions to describe PDE operators and applies compiler transformations to generate optimized native code. Compared with Yasmin, Devito starts from a higher-level mathematical abstraction, whereas Yasmin begins with an already discretized stencil operator.

Pystencils is a Python-based code-generation framework for stencil computations that generates optimized kernels for CPUs and GPUs [4]. Its use in large-scale phase-field simulations demonstrates a similar separation between high-level stencil descriptions and architecture-specific generated code.

**Contribution.** In contrast to these mature systems, Yasmin is intentionally smaller in scope. Its main contributions are a compact embedded Python DSL for expressing stencil computations, a two-level intermediate representation consisting of a semantic Stencil IR and an explicit Loop IR, and three execution paths targeting NumPy, C++, and

## 2. YASMIN DESIGN

This separation keeps the frontend close to the mathematical stencil while making backend-specific execution decisions explicit and inspectable. In particular, the distinction between Stencil IR and Loop IR allows Yasmin to reuse the same semantic representation across different execution strategies without forcing all backends through the same lowering path. Figure 1 summarizes this pipeline.

## 2.1. Frontend

Yasmin provides an embedded Python DSL for expressing stencil computations. The frontend is based on symbolic objects and operator overloading, allowing stencil expressions to be written using regular Python syntax while building an internal representation of the computation. Fields are indexed using relative offsets rather than absolute grid coordinates. For example, the two-dimensional five-point Laplacian can be written as

```
import yasmin as yasi

@yasi.operator
def laplacian(u, out):
    out[0, 0] = (
        u[-1, 0]
        + u[1, 0]
        + u[0, -1]
        + u[0, 1]
        - 4.0 * u[0, 0]
    )
```

```

@yasi.operator
def laplacian(u, out):
    out[0, 0] = (
        u[-1, 0]
        + u[1, 0]
        + u[0, -1]
        + u[0, 1]
        - 4.0 * u[0, 0]
    )
  
```

```

Operator(statements=(Assign(
    target=FieldAccess(out, (0,)),
    value=BinaryExpr(
        BinaryOp.ADD,
        BinaryExpr(
            BinaryOp.SUB,
            FieldAccess(u, (0,)),
            BinaryExpr(
                BinaryOp.MUL,
                Literal(2),
                FieldAccess(u, (1,)),
            ),
        ),
        FieldAccess(u, (2,))),
),),),),
)
  
```

**Stencil IR**

```

Function(name='laplacian',
        fields=(Field(name='out', ...),
                Field(name='u', ...)),
        body=(For(
            index=Index(name='x'),
            lower=Literal(0),
            upper=BinaryExpr(
                BinaryOp.SUB, Extent(
                    field=Field(name='out', ...),
                    Literal(2),
                ),
            body=(Store(Field(name='out', ...),
                        indices=(Index(name='x'),),
  
```

**Loop IR**

```

extern "C" void laplacian(double *u, ...) {
    for (long i0 = 1; i0 < n0 - 1; ++i0) {
        for (long i1 = 1; i1 < n1 - 1; ++i1) {
            const long idx = i0 * n1 + i1;
            out[idx] =
                u[(i0 - 1) * n1 + i1] +
                u[(i0 + 1) * n1 + i1] +
                u[i0 * n1 + (i1 - 1)] +
                u[i0 * n1 + (i1 + 1)] -
                4.0 * u[idx];
        }
    }
}
  
```

**C++**

```

extern "C" void laplacian(double *u, ...) {
    #pragma omp parallel for collapse(2)
    for (long i0 = 1; i0 < n0 - 1; ++i0) {
        for (long i1 = 1; i1 < n1 - 1; ++i1) {
            const long idx = i0 * n1 + i1;
            out[idx] =
                u[(i0 - 1) * n1 + i1] +
                u[(i0 + 1) * n1 + i1] +
                u[i0 * n1 + (i1 - 1)] +
                u[i0 * n1 + (i1 + 1)] -
                4.0 * u[idx];
        }
    }
}
  
```

**OpenMP**

**compilation**

**Native Artifact**

**ctypes invocation**

**Fig. 1.** Yasmin compilation and execution pipeline. The Python frontend constructs a semantic Stencil IR. NumPy executes directly from this representation, while native backends lower through an explicit Loop IR before C++ or OpenMP code generation and compilation.

the output value at the current grid point. The frontend therefore describes a stencil relative to the point currently being evaluated without exposing explicit loop indices or memory addresses.

The frontend distinguishes between logical dimensions, fields, scalar values, and symbolic expressions. Arithmetic on these objects constructs expression trees instead of immediately performing the corresponding numerical operation. The `@yasi.operator` decorator groups one or more such assignments into a reusable stencil operator.

Operators may contain multiple assignments. Their semantics are sequential: each assignment is completed before the following assignment is evaluated. This is relevant when a later assignment depends on values written by an earlier one.

Yasmin only captures the stencil computation itself. Regular Python remains responsible for control flow, array allocation, initializing fields, configuring a backend, invoking kernels, and implementing larger application logic. This keeps the DSL small while allowing it to remain embedded in a general-purpose programming environment.

## 2.2. Stencil IR

The frontend constructs a backend-independent intermediate representation, referred to as the *Stencil IR*. Its purpose is to describe what a stencil computes without specifying how the computation is executed. The Stencil IR contains assignments, literals, arithmetic expressions, and field accesses. A field access stores a relative offset for every logical dimension. For the Laplacian above, the relevant accesses are therefore `u@(-1, 0)`, `u@(1, 0)`, `u@(0, -1)`, `u@(0, 1)`, and `u@(0, 0)`. Conceptually, part of the representation can be visualized as

---

```
Assign(
    target=FieldAccess(out, (0, 0)),
    value=BinaryExpr(SUB,
        BinaryExpr(ADD,
            BinaryExpr(ADD,
                BinaryExpr(ADD,
                    FieldAccess(u, (-1, 0)),
                    FieldAccess(u, (1, 0)),
                ),
                FieldAccess(u, (0, -1)),
            ),
            FieldAccess(u, (0, 1)),
        ),
        BinaryExpr(MUL,
            Literal(4.0),
            FieldAccess(u, (0, 0)),
        ),
    ),
)
```

---

At this level, there are no explicit loop variables, loop bounds, flattened array indices, or backend-specific constructs. This keeps the representation close to the mathematical stencil and makes it independent of whether it is later executed using NumPy, C++, or OpenMP.

The representation is generic in dimensionality rather than being restricted to the two-dimensional kernels evaluated in this report. Field data types are also preserved in the IR. The current implementation supports `float32`, `float64`, `int32`, and `int64`.

Logical dimensions and physical field shapes are deliberately kept separate. Fields that share the same logical dimensions do not necessarily need to have identical physical extents. This allows the representation to express, for example, fields located at different positions on a staggered grid. The concrete field shapes are taken into account later when deriving the valid execution domain.

## 2.3. Analysis and Lowering

Before native code can be generated, Yasmin analyzes the Stencil IR to determine where each assignment can be evaluated safely. The analysis collects the field accesses appearing in an expression and extracts their relative offsets, which determine the halo required by the stencil.

For example, the Laplacian accesses one neighboring element in both directions of each dimension. If an input field has shape  $(N_x, N_y)$ , the resulting execution domain is

$$1 \leq i < N_x - 1, \quad 1 \leq j < N_y - 1.$$

The same analysis generalizes to larger offsets and to  $N$  dimensions. The execution domain is inferred from both the stencil offsets and the concrete shapes of all participating fields. Consequently, fields sharing a logical dimension may have different physical extents while Yasmin still derives a valid common execution region.

After the valid domain has been determined, the Stencil IR is lowered to the Loop IR. Lowering introduces explicit iteration variables and replaces relative accesses with expressions in terms of those variables. A semantic access such as `u[-1, 0]` therefore becomes conceptually `u[i0 - 1, i1]`.

Each assignment is currently lowered independently. An operator containing multiple assignments therefore results in multiple sequential loop nests. Automatic loop fusion is not currently performed.

## 2.4. Loop IR

The *Loop IR* represents the computation after the iteration structure has been made explicit. It contains loop variables, loop bounds, indexed field accesses, arithmetic expressions, and assignments. For the Laplacian example, the lowered representation corresponds conceptually to

---

```

Function(
  name='laplacian',
  fields=(Field('out', ...), Field('u',
    ↪ ...)),
  body=(
    For(x, 1, Extent(out, 0)-1,
      For(y, 1, Extent(out, 1)-1,
        Store(out[x,y],
          u[x-1,y] + u[x+1,y] + u[x,y-1] +
            ↪ u[x,y+1]
          - 4.0*u[x,y]))),
    ...
  )
)

```

---

The Loop IR remains independent of a particular source language, but is close enough to conventional loops to be consumed by both the C++ and OpenMP backends. Multidimensional field accesses remain independent of their final memory layout; during C++ code generation they are translated to row-major linear accesses. For a two-dimensional field with second extent  $N_1$ , the access  $(i_0, i_1)$  maps to  $i_0N_1 + i_1$ .

## 2.5. Execution Backends

Yasmin currently supports three execution backends: NumPy, C++, and C++ with OpenMP. The NumPy backend operates directly on the Stencil IR, while the two native backends use the explicit Loop IR introduced during lowering.

### 2.5.1. NumPy

The NumPy backend executes the stencil directly from the Stencil IR using array slicing. Relative field accesses are translated into slices over the valid execution domain, and the resulting expressions are evaluated using NumPy operations. Since NumPy already performs the iteration internally, lowering to the Loop IR would introduce unnecessary explicit loops. The NumPy backend therefore deliberately bypasses the Loop IR and native compilation.

### 2.5.2. C++

The C++ backend operates on the Loop IR, where iteration domains and field accesses have already been made explicit. It generates nested loops over the valid execution domain and translates multidimensional field accesses into row-major memory accesses. The generated function operates directly on typed pointers to the underlying NumPy buffers. The code generation itself is kept deliberately simple so that the generated source remains easy to inspect.

**Pointer Aliasing.** Because stencil computations inherently operate on distinct input and output buffers, Yasmin can annotate them with `__restrict__`. This keyword promises

the compiler that the buffers do not alias, enabling more aggressive optimizations such as vectorization.

### 2.5.3. OpenMP

The OpenMP backend extends the generated C++ code with OpenMP parallelization. A basic configuration parallelizes the outermost loop with `#pragma omp parallel for`; Yasmin additionally supports shape-aware decisions to avoid parallelization configurations that are inefficient for small or narrow iteration domains.

**Shape-Aware Code Generation.** Parallelization of the code is only beneficial if there is enough independent work. Additionally, since parallelization introduces overhead, we need to make sure that the problem size is big enough to achieve a speed-up. For this reason, Yasmin resolves the concrete field shapes before code generation and folds each loop bound against them to get its exact trip count. The thread count and the collapsing decision below are both derived from this trip count instead of a static guess.

**Number of Threads.** Using too many threads for a small iteration domain can make parallel overhead dominate useful work. Yasmin therefore uses a threshold, currently set to 1000 iterations per thread, to determine how many threads should be requested. If the total amount of work falls below this threshold, OpenMP pragmas are omitted entirely to avoid unnecessary parallel overhead.

**Loop Collapsing.** The 2D loop nests currently generated by Yasmin are rectangular, which makes `collapse(2)` structurally valid. 1D stencils have only a single loop, so collapsing does not apply to them. However, collapsing introduces overhead and is only justified when the outer loop alone cannot saturate the available threads, but the combined loop nest can. Yasmin therefore uses the trip count to calculate when this is the case and enables `collapse(2)` only then.

**Scheduling.** For the uniform per-point cost of the Laplacian, `schedule(static)` without an explicit chunk size is preferable to any chunked alternative since with no chunk given, OpenMP splits the iteration space into exactly `num_threads` contiguous blocks, giving each thread one solid, cache-friendly range. Yasmin also supports `schedule(dynamic)` and `schedule(guided)` for future, non-uniform-cost stencils.

## 2.6. Compilation and Runtime

The native backends require an additional compilation and invocation step between code generation and execution. Yasmin keeps this process explicit so that both the generated source code and the configuration used to generate it remain inspectable.

**Compilation Configuration.** Compilation is controlled through a backend configuration that specifies the relevant

compiler and code-generation options. For OpenMP, this includes options such as the number of threads, scheduling policy, loop collapsing, and whether adaptive decisions should be used. Shape-dependent options are resolved before code generation, resulting in a concrete configuration for the kernel being compiled.

**Native Compilation.** For the C++ and OpenMP backends, Yasmin first generates C++ source code from the Loop IR. The source is then compiled into a native shared library using the configured C++ compiler. The OpenMP backend additionally passes `-fopenmp` to the compiler. Code generation and compilation are separate from kernel execution and can therefore be performed once and reused for multiple invocations.

**Kernel Invocation.** The compiled shared library is loaded from Python using `ctypes`. NumPy arrays supplied by the user are mapped to the native pointer arguments expected by the generated function. Before native execution, Yasmin validates the bindings, including the field shapes and data types, so that invalid arguments are rejected before crossing the native interface.

**Compile and Execute.** Yasmin distinguishes between explicit compilation and direct execution. The `compile()` interface produces a reusable `Kernel` object that can be invoked multiple times with different compatible field bindings. The `execute()` interface provides a convenience path for users who want to compile and execute an operator directly.

This distinction is useful when the same stencil is executed repeatedly, since the cost of compilation does not need to be paid for every invocation.

**Compilation Cache.** To avoid recompiling identical kernels, Yasmin maintains an in-process compilation cache. Both `compile()` and `execute()` use the same cache, which allows an already compiled native kernel to be reused when the same configuration is encountered again. The current cache exists only for the lifetime of the Python process; a persistent on-disk compilation cache is left as future work.

**Kernel Inspection.** The generated source code and the resolved backend configuration can be inspected through the compiled kernel. This makes decisions such as the chosen OpenMP thread count, loop collapsing, scheduling policy, loop bounds, and generated memory accesses visible to the user. Kernel inspection therefore provides a concrete form of performance transparency. The abstraction does not hide how a stencil is eventually mapped to native execution.

## 2.7. Scope and Limitations

Yasmin intentionally implements a relatively small stencil abstraction rather than attempting to provide the full functionality of mature stencil frameworks. The current implementation supports generic  $N$ -dimensional stencils, relative field accesses, multiple data types, and fields with different

physical extents. The latter allows fields that share the same logical dimension to represent, for example, basic staggered-grid layouts. Operators may contain multiple assignments, which are executed sequentially.

Several higher-level features found in larger frameworks remain outside the current scope. This is deliberate, since the focus here is a short and inspectable path from a Python stencil expression to NumPy, C++, and OpenMP execution. Possible extensions are discussed in Section 6.

## 3. EXPERIMENTAL METHODOLOGY

In this and the next section, we discuss Yasmin’s performance capabilities through two examples with applications in weather and climate models. We test all Yasmin backends against comparable hand-written implementations on CSCS’s Alps cluster, considering both runtime and scaling capabilities. We also reproduce the same examples using GT4Py, which serves as an established stencil-DSL baseline.

### 3.1. Workloads and Baselines

The first example is a simple approximation of the two-dimensional Laplacian operator ( $\nabla^2$ ) on a regular grid using finite differences, which results in the 5-point stencil

$$\begin{aligned} \text{out}[i, j] = & -4 * \text{in}[i, j] \\ & + \text{in}[i + 1, j] + \text{in}[i - 1, j] \\ & + \text{in}[i, j + 1] + \text{in}[i, j - 1]. \end{aligned}$$

Here, `out` and `in` are fields over the grid, and  $i$  and  $j$  are indices in the  $x$  and  $y$  direction, respectively. The stencil is applied once to the interior points of an input field initialized with a sinusoidal function, and the result is saved in an output field. Its simplicity makes it useful for exposing framework and optimization overhead.

The second example is based on the two-dimensional viscous Burgers’ equations:

$$\begin{aligned} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} &= \mu \left( \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \\ \frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} &= \mu \left( \frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right), \end{aligned}$$

with velocity fields  $u$  and  $v$  and viscosity  $\mu$ . Applying a fifth-order upwind advection scheme and fourth-order centered diffusion scheme results in four separate stencil computations, two for advection and two for diffusion. The time derivative can be resolved with a Runge–Kutta method.<sup>1</sup>

<sup>1</sup>More information about the problem can be found in the GT4Py Burgers example: [https://github.com/GridTools/gt4py/blob/main/examples/cartesian/demo\\_burgers.ipynb](https://github.com/GridTools/gt4py/blob/main/examples/cartesian/demo_burgers.ipynb), on which this benchmark is largely based.

For the benchmark presented here, we consider one application of the spatial operator; the surrounding time-integration loop is excluded. This ensures that Yasmin, the hand-written implementations, and GT4Py perform comparable stencil-kernel work. Compared with the Laplacian, this workload contains more complex stencil expressions and is substantially more computationally intensive.

We compare three groups of implementations: NumPy, GT4Py with its NumPy backend (`gt4py_numpy`), and Yasmin with its NumPy backend (`yasmin_numpy`); a single-core C++ implementation, GT4Py with its CPU backend (`gt4py_cpu`), and Yasmin with its C++ backend (`yasmin_cpp`); and multi-core implementations consisting of C++ with OpenMP, GT4Py’s CPU backend, and Yasmin’s OpenMP backend (`yasmin_openmp`). For the scaling experiments, only these three parallel implementations are considered.

### 3.2. Experimental Setup

All benchmarks are conducted on CSCS’s Alps cluster using one NVIDIA Grace CPU with 72 available cores. The software environment uses Python 3.14.3 and GCC 14.3.0.

For the OpenMP experiments, dynamic thread adjustment is disabled and threads are bound to cores using

```
OMP_DYNAMIC=FALSE
OMP_PROC_BIND=close
OMP_PLACES=cores
```

Thread counts are set explicitly rather than relying on implementation-specific defaults. The runtime comparison uses 8 threads for the multi-core implementations, while the scaling experiments use 1, 2, 4, 8, 16, 32, and 64 threads. GT4Py’s CPU backend is additionally executed with one thread for the single-core comparison.

The benchmark log records the source-code commit, Python and compiler versions, and OpenMP environment. For all configurations, correctness is checked against the NumPy reference implementation before a timing result is accepted.

### 3.3. Runtime and Scaling Methodology

For the runtime comparison, both workloads are evaluated on square grids with side lengths 127, 255, 511, 1023, 2047, 4095, 8191, corresponding to  $2^k - 1$ . This preserves an approximately geometric progression of problem sizes while avoiding exact power-of-two row strides and reducing the risk of systematic memory-layout effects.

Strong scaling is evaluated on a fixed  $8191 \times 8191$  grid while doubling the number of threads from 1 to 64. Speedup is calculated independently for each implementation as

$$S_i(p) = \frac{T_i(1)}{T_i(p)},$$

where  $T_i(p)$  denotes the runtime of implementation  $i$  using  $p$  threads.

For weak scaling, the amount of work is increased proportionally with the number of threads  $p$ . Instead of scaling both dimensions, which would require side lengths proportional to  $\sqrt{p}$  and therefore rounding for several of the tested thread counts, we keep the first dimension fixed and extend only the second:

$$n_x = 1023, \quad n_y = 1023p.$$

This gives exactly  $1023 \times 1023$  nominal grid points per thread for every tested value of  $p$ , while also avoiding irregular domain sizes introduced by a square  $\sqrt{p}$  construction. Weak-scaling efficiency is defined as

$$E_i(p) = \frac{T_i(1)}{T_i(p)} \cdot 100\%.$$

For the advection–diffusion workload, the grid spacing is kept fixed during weak scaling,

$$\Delta x = \frac{1}{n_x - 1}, \quad \Delta y = \Delta x,$$

so increasing  $n_y$  extends the physical domain rather than changing the spatial resolution.

For all benchmarks, code generation and compilation time are excluded such that only kernel execution is measured. Three warm-up runs are performed before 15 measured repetitions, and the median execution time is reported to reduce the influence of transient effects and system noise. Python implementations use `time.perf_counter()`, while the C++ implementations use `std::chrono::high_resolution_clock`.

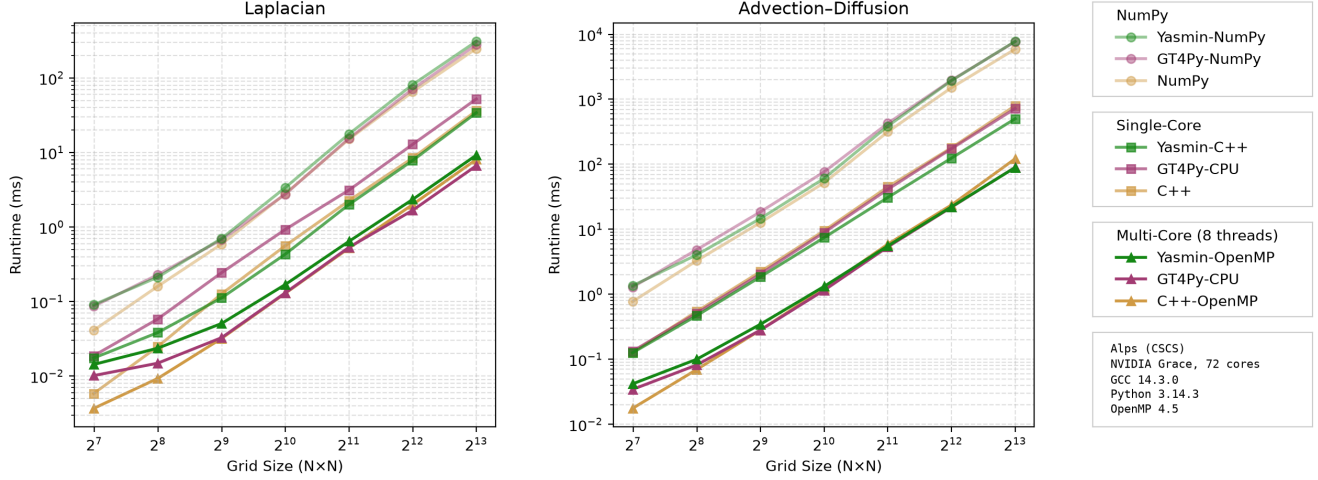
## 4. EVALUATION

### 4.1. Runtime and Abstraction Cost

Figure 2 compares the absolute kernel runtime of the NumPy, single-core, and 8-thread multi-core implementations. For the NumPy execution path, Yasmin introduces a visible overhead compared with hand-written NumPy, particularly for the simpler Laplacian kernel. GT4Py-NumPy and Yasmin-NumPy remain in a similar performance range, while the direct NumPy implementation is generally faster.

For the native backends, the differences are considerably smaller. Yasmin-C++ remains competitive with both hand-written C++ and GT4Py’s single-threaded CPU backend across the tested problem sizes. The same holds for Yasmin-OpenMP in the 8-thread comparison, where its runtime stays close to the hand-written OpenMP and GT4Py





**Fig. 2.** Runtime versus grid size for the Laplacian and advection–diffusion kernels. Results compare Yasmin against handwritten NumPy/C++/OpenMP implementations and GT4Py. The multi-core measurements use 8 threads.

implementations. For the larger advection–diffusion cases, Yasmin-C++ even achieves lower measured runtimes than the hand-written C++ baseline; however, we do not attribute this to a specific compiler optimization without further analysis.

The results indicate that Yasmin’s abstraction cost depends on the execution path. The NumPy backend shows measurable overhead, whereas the C++ and OpenMP paths show performance close to the corresponding low-level implementations.

## 4.2. Strong Scaling

Figure 4 shows the strong-scaling behavior of the three parallel implementations on the fixed  $8191 \times 8191$  domain. For the Laplacian, all implementations follow the ideal trend closely up to about 16 threads, after which scaling begins to saturate. At 64 threads, Yasmin-OpenMP reaches roughly  $30\times$  speedup, compared with the ideal  $64\times$ .

The advection–diffusion kernel scales substantially better. All three implementations remain close to ideal scaling through 32 threads and still achieve approximately  $53\text{--}61\times$  speedup at 64 threads. The reasons for this difference in scaling behavior are discussed in Section 5.

The increasing deviation from ideal scaling at high thread counts is consistent with Amdahl’s law and with growing parallel-runtime and memory-system overheads. The measurements alone, however, do not identify which of these effects dominates.

## 4.3. Weak Scaling

Figure 3 shows the weak-scaling efficiency when the problem size is increased proportionally with the number of

threads. For the Laplacian, Yasmin-OpenMP remains close to ideal efficiency up to 16 threads, before dropping to about 74% at 32 threads and about 26% at 64 threads. The hand-written OpenMP and GT4Py implementations show a similar decline at the largest thread counts.

The advection–diffusion workload exhibits better weak scaling overall. Yasmin-OpenMP remains above roughly 87% efficiency through 32 threads, but drops to about 42% at 64 threads. GT4Py behaves differently at the largest thread count and retains substantially higher efficiency, indicating that the high-thread-count degradation is not solely determined by the hardware.

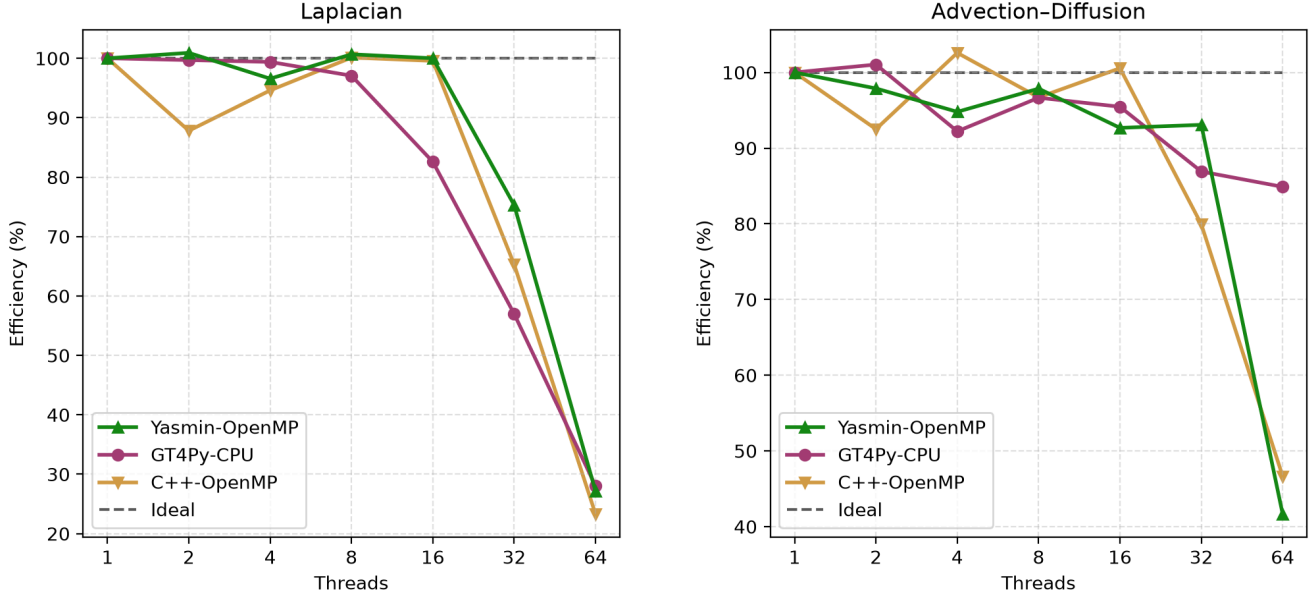
Overall, Yasmin maintains near-ideal weak scaling over the low and moderate thread counts for both workloads, while efficiency decreases significantly at 64 threads.

## 4.4. Readability, Portability, and Transparency

Beyond runtime performance, Yasmin also aims to reduce implementation complexity. For example, the five-point Laplacian can be expressed directly using relative field accesses:

```
@yasi.operator
def laplacian(u, out):
    out[0, 0] = (
        u[-1, 0] + u[1, 0]
        + u[0, -1] + u[0, 1]
        - 4.0 * u[0, 0]
    )
```

The same stencil definition can be executed using the NumPy, C++, and OpenMP backends without rewriting the numerical expression. In contrast, the generated native imple-



**Fig. 3.** Weak-scaling efficiency with a fixed  $1023 \times 1023$  workload per thread. The global domain is  $1023 \times (1023 \cdot p)$  for  $p$  threads.

mentation makes loop bounds, indexing, and parallelization explicit. A corresponding OpenMP kernel contains code of the form

```
#pragma omp parallel for collapse(2)
for (int i0 = 1; i0 < n0 - 1; ++i0)
    for (int i1 = 1; i1 < n1 - 1; ++i1)
        out[i0 * n1 + i1] = ...;
```

This illustrates the separation between numerical intent and execution strategy: the user-facing stencil remains close to its mathematical form, while backend-specific details are introduced automatically.

At the same time, these details are not hidden from the user. Yasmin exposes the Stencil IR and Loop IR, the generated C++ or OpenMP source, and the resolved backend configuration. The transformation from a high-level stencil expression to explicit loops and parallelization decisions can therefore be inspected directly, providing performance transparency in addition to readability and portability.

## 5. DISCUSSION

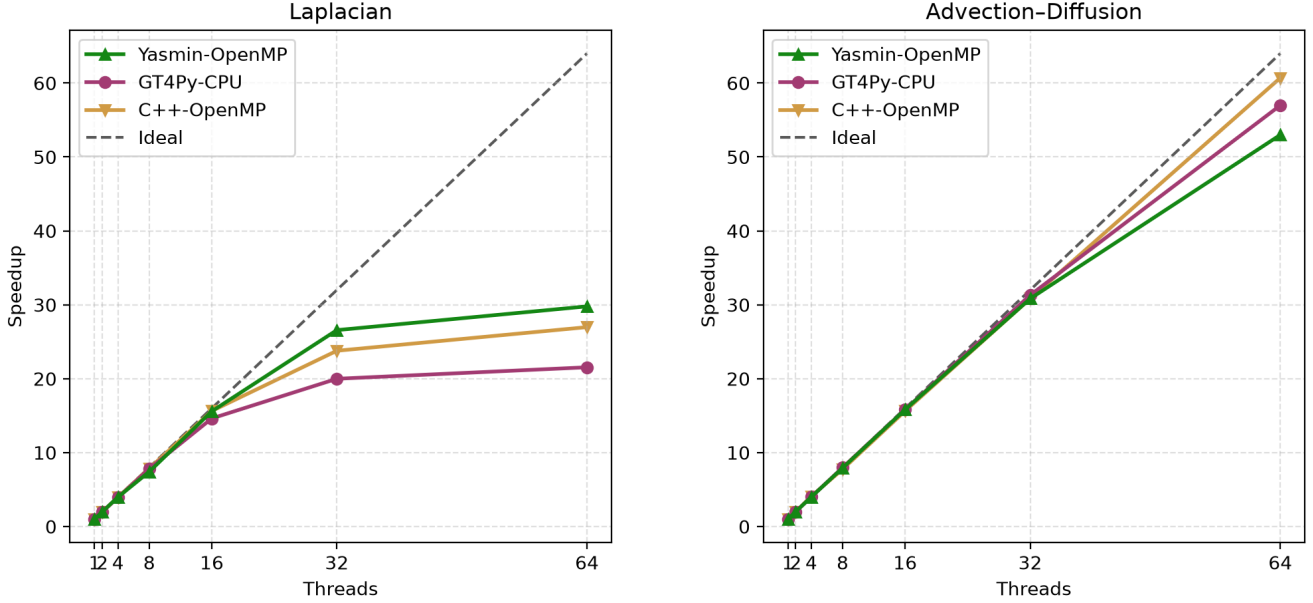
**Abstraction Cost.** The evaluation shows that the cost of the Yasmin abstraction depends strongly on the execution path. The additional overhead is most visible for the NumPy backend, particularly for the comparatively simple Laplacian. In contrast, the generated C++ and OpenMP kernels remain close to the hand-written implementations. This suggests that the semantic abstraction itself does not necessarily im-

ply a substantial runtime penalty once it has been lowered to explicit native loops. In some configurations, Yasmin even outperforms the hand-written C++ baseline. While this demonstrates that generated code can be competitive, the measurements alone do not establish the reason for these differences; a detailed analysis of generated machine code would be required.

**Parallel Scaling.** The different scaling behavior of the two workloads illustrates that parallel performance depends not only on the backend but also on the computational characteristics of the stencil. The relatively simple Laplacian saturates earlier, whereas the more computationally intensive advection-diffusion kernel continues to benefit from additional cores. This behavior is consistent with the Laplacian placing greater pressure on memory movement relative to useful computation, although confirming the underlying bottleneck would require hardware performance counters. At high thread counts, synchronization, OpenMP runtime overhead, and the memory hierarchy can all contribute to deviations from ideal scaling. The comparatively strong weak-scaling result of GT4Py for advection-diffusion at 64 threads also shows that the observed degradation cannot be attributed solely to a common hardware limit.

**Design Trade-offs.** Yasmin deliberately favors a small and transparent compiler pipeline over a large set of domain abstractions and compiler optimizations. The two-level IR and separate execution backends are sufficient to provide portability across NumPy, C++, and OpenMP while keeping the transformation from a stencil expression to generated





**Fig. 4.** Strong scaling on a fixed  $8191 \times 8191$  domain. Speedup is normalized independently to each implementation’s one-thread runtime. The dashed line denotes ideal linear scaling.

code inspectable. The cost of this simplicity is the absence of features available in more mature stencil frameworks, such as automatic loop fusion, explicit boundary-condition abstractions, time stepping, and GPU execution. The results therefore support the viability of the core design rather than establishing feature parity with systems such as GT4Py.

**Evaluation Limitations.** The evaluation is limited to two two-dimensional stencil kernels, one CPU architecture, one compiler toolchain, and a single compute node. The results therefore cannot establish performance portability across architectures such as GPUs or different CPU systems. Furthermore, the advection–diffusion benchmark evaluates the spatial kernel rather than the complete time-dependent simulation. Compilation and code-generation costs are deliberately excluded, so the reported results characterize steady-state kernel execution rather than first-call or end-to-end application performance. Finally, runtime and scalability alone do not identify architectural bottlenecks; measurements of memory bandwidth, cache behavior, vectorization, and generated assembly would be required to explain some of the observed performance differences in more detail.

## 6. OUTLOOK

Several extensions to Yasmin remain natural next steps. On the level of program semantics, the execution domain currently cannot be specified by the user, boundary conditions must be handled in surrounding Python code rather than within the DSL, and time-stepping is expressed as an outer

Python loop rather than as part of a Yasmin operator. Adding explicit constructs for these three would let a stencil definition specify its own execution domain, boundary conditions, and time-stepping directly.

On the optimization side, each assignment is currently lowered into its own loop nest, and Yasmin does not fuse them. Doing so would require analyzing dependencies between assignments to decide when two loop nests can be safely merged, which we leave as future work.

On the execution side, adding a CUDA or CuPy backend and support for distributed-memory execution via halo exchange would let Yasmin target the GPU clusters used in real weather and climate forecasting. A higher-level symbolic frontend, possibly with automatic differentiation, would give Yasmin capabilities similar to frameworks like Devito [3].

Finally, the current compilation cache only lives for the duration of a Python process. Making it persist on disk would avoid recompiling the same kernel across separate runs.

## 7. CONCLUSION

We presented Yasmin, a compact embedded Python DSL for structured-grid stencil computations. Yasmin separates the numerical stencil definition from its execution strategy through a semantic Stencil IR and an explicit Loop IR, targeting NumPy, C++, and OpenMP backends while keeping the compilation pipeline inspectable.

Our evaluation shows that the NumPy backend introduces some abstraction overhead, whereas the generated C++

and OpenMP backends achieve performance close to comparable hand-written implementations and GT4Py. Yasmin shows good parallel scaling over low and moderate thread counts, with the more computationally intensive advection–diffusion kernel scaling more effectively to higher thread counts.

Yasmin deliberately remains small in scope. Its main result is therefore not feature completeness, but that a compact and transparent stencil abstraction can provide portability across multiple CPU execution strategies while retaining competitive native performance.

## 8. REFERENCES

- [1] Johannes Pekkilä, Miikka S Väisälä, Maarit J Käpylä, Matthias Rheinhardt, and Oskar Lappi, “Scalable communication for high-order stencil computations using CUDA-aware MPI,” *Parallel Comput.*, vol. 111, no. 102904, pp. 102904, July 2022.
- [2] Enrique G Paredes, Linus Groner, Stefano Ubbiali, Hannes Vogt, Alberto Madonna, Kean Mariotti, Felipe Cruz, Lucas Benedicic, Mauro Bianco, Joost VandeVondele, and Thomas C Schulthess, “GT4Py: High performance stencils for weather and climate applications using python,” Nov. 2023.
- [3] Mathias Louboutin, Michael Lange, Fabio Luporini, Navjot Kukreja, Philipp A Witte, Felix J Herrmann, Paulius Velesko, and Gerard J Gorman, “Devito (v3.1.0): an embedded domain-specific language for finite differences and geophysical exploration,” *Geosci. Model Dev.*, vol. 12, no. 3, pp. 1165–1187, Mar. 2019.
- [4] Martin Bauer, Johannes Hötzer, Dominik Ernst, Julian Hammer, Marco Seiz, Henrik Hierl, Jan Hönig, Harald Köstler, Gerhard Wellein, Britta Nestler, and Ulrich Rüde, “Code generation for massively parallel phase-field simulations,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, New York, NY, USA, Nov. 2019, pp. 1–32, ACM.